Course Outline: Introduction to Data Pipelines

Title: Introduction to Data Pipelines **Skill:** Skill 39 — Building a data pipeline from the telemetry or user data **Learnpack:** + Introducción a Data Pipelines **File:** s39-w14d40-introduccion-a-data-pipelines **Prerequisite:** How Data Flows in a System **Target Audience:** Beginner developers building their first data-aware applications **Total Lessons:** 11 **Format:** Mixed (9% hands-on)

Course Description

You already know how data moves through a system — from sources to ingestion to transformation to storage. Now it's time to formalize that flow into something reliable, repeatable, and automatic: a data pipeline. This course breaks down the ETL pattern, the components that implement it, and gives you hands-on practice building a basic pipeline in Python using FastAPI, pandas, and numpy.

Course Philosophy

This course emphasizes:

- Connecting theory to practice — every concept leads directly to code
- Precision over breadth — understand fewer things deeply rather than many things shallowly
- Making mistakes explicit — knowing what can go wrong is as important as knowing the right approach

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The Case for Pipelines	3
02 - ETL and Pipeline Components	5
03 - Assessment	1
04 - Outro	1
Total	**11**

00.0 Welcome 🕮

Learning Objectives:

- Understand what this course covers and how it connects to the previous learnpack
- Identify the two main sections of the course and what each delivers

Content Outline:

- Brief recap: in the previous learnpack, students learned how data flows through a system (sources → ingestion → transformation → storage)
- The gap: that flow exists, but who runs it? Who ensures it happens reliably, every time, at scale?
- Answer: a data pipeline
- Course preview: Section 01 builds the case for pipelines — what they are, why automation matters, and the batch vs. stream distinction; Section 02 dives into the ETL pattern and the three components that implement it, ending with a hands-on Python challenge

Transitions: This lesson sets expectations. The next lesson formally defines a data pipeline and positions it within what students already know.

Key Principles:

- Data pipelines are the automation layer on top of data flow
- Understanding why they exist makes the implementation obvious

Examples:

- A daily sales report that used to require a human exporting a CSV and uploading it manually — now runs on its own every night
- A real-time fraud detection system that processes transactions as they arrive instead of in overnight batches

---

01.0 Data Pipelines 🕮

Learning Objectives:

- Define a data pipeline in concrete terms
- Distinguish a pipeline from ad-hoc data scripts
- Explain how pipelines relate to the data flow concepts learned in the previous learnpack

Content Outline:

- Definition: a data pipeline is an automated, structured sequence of steps that moves data from one or more sources to one or more destinations, applying transformations along the way
- Key characteristics: automated (no manual trigger), repeatable (runs the same way every time), observable (you can see what happened and when)
- How it extends the previous learnpack: the stages of data flow (ingest, transform, store) are exactly what a pipeline automates — the pipeline is the mechanism that connects and orchestrates those stages
- Types of pipelines at a glance: batch, stream, micro-batch (introduced briefly; covered in depth in lesson 01.2)

Transitions: Now that the concept is clear, the next lesson makes the case for why automating data processing is not optional.

Key Principles:

- A pipeline is not a script — it is a structured system with defined inputs, outputs, and stages
- Observability is a first-class property: a pipeline you cannot monitor is a pipeline you cannot trust

Examples:

- An e-commerce pipeline that collects purchase events, cleans them, and loads them into an analytics database every hour
- A machine learning pipeline that pulls user activity data, computes features, and feeds a recommendation model

---

01.1 Why Automate Data Processing 🕮

Learning Objectives:

- Articulate the specific failure modes of manual data processing
- Explain what automation guarantees that manual work cannot
- Identify the key properties of a reliable automated pipeline

Content Outline:

- The before picture: manual data processing in practice — someone runs a script, exports a CSV, sends it by email, another person imports it
- What goes wrong: steps get skipped, formats change, the person is sick, the script runs twice, data is stale
- The failure modes in concrete terms: data loss, duplication, inconsistency, non-reproducibility, zero observability
- What automation adds: scheduled execution, error handling, retry logic, logging, idempotency
- Scale argument: at 100 records per day, manual is inconvenient; at 100 million, it is impossible
- Best practices (woven in): always log what ran and when; design pipelines to be idempotent (running twice should produce the same result as running once); separate pipeline logic from ad-hoc analysis
- Anti-patterns (woven in): cron jobs without error handling ("fire and forget"); no alerting when a pipeline fails; tightly coupling pipeline steps so one failure breaks everything

Transitions: Automation is necessary — but how do you structure what gets automated? The next lesson answers that with the ETL pattern, which is the backbone of most data pipelines.

Key Principles:

- Manual data processing does not scale and cannot be trusted
- Automation is not about convenience — it is about correctness and reliability
- Idempotency is the foundation of trustworthy pipeline design

Examples:

- A pipeline that sends duplicate records to a database because it was run twice without idempotency — and how a simple deduplication key prevents it
- A nightly batch job that fails silently because there is no alerting, and the team only notices three days later when reports are wrong

01.2 Batch vs. Stream 🕮

Learning Objectives:

- Define batch processing and stream processing and their differences
- Choose the appropriate processing mode for a given scenario
- Recognize the tradeoffs between latency and throughput

Content Outline:

- Two fundamental modes: batch and stream
- Batch: data is collected over a period and processed all at once at a scheduled time
  - Characteristics: high throughput, higher latency, simpler to build
  - Use cases: end-of-day reports, weekly model retraining, data warehouse loads
- Stream: data is processed continuously as it arrives, event by event
  - Characteristics: low latency, more complex, requires a streaming infrastructure
  - Use cases: real-time fraud detection, live dashboards, telemetry event processing
- Micro-batch: a middle ground — processes data in small batches every few seconds (e.g., Spark Structured Streaming)
- The tradeoff: you almost never need real-time when near-real-time will do; batch is almost always simpler and cheaper when latency is not critical
- Common mistakes (woven in): choosing streaming because it sounds more impressive when batch would be sufficient; not accounting for the operational complexity of stream infrastructure; treating all data as equally time-sensitive

Transitions: With the motivation and processing modes established, Section 02 begins by introducing the ETL pattern — the canonical way to structure a data pipeline regardless of whether it runs in batch or stream mode.

Key Principles:

- Most pipelines are batch — and that is perfectly fine
- The right mode is determined by how fresh the data needs to be, not by what sounds modern
- Streaming adds complexity; that complexity must be justified by the latency requirement

Examples:

- A fraud detection system at a payment processor — stream processing is justified because each transaction must be evaluated before it completes
- A weekly business intelligence report — batch is the right choice; nobody needs it refreshed every second
- An e-commerce recommendation engine — a micro-batch refresh every 5 minutes is a pragmatic middle ground

---

02.0 ETL 🕮

Learning Objectives:

- Define ETL and explain what each stage does
- Explain why the ETL sequence is ordered the way it is
- Map ETL stages to the data flow concepts learned in the previous learnpack
- Preview how each stage corresponds to a dedicated component in the pipeline

Content Outline:

- ETL stands for Extract, Transform, Load — the canonical three-stage pattern for data pipelines
- Why this order matters: you extract first (pull raw data as-is), transform second (clean and reshape it in memory), and load last (write clean data to the destination) — doing it in any other order introduces data quality problems at the destination
- Extract: pulling data from one or more sources — databases, APIs, files, streams
- Transform: applying logic to the extracted data — cleaning nulls, normalizing formats, computing derived fields, filtering invalid records
- Load: writing the transformed data to its destination — a database, a data warehouse, a file, an API
- Connection to previous learnpack: ingestion ≈ Extract; transformation ≈ Transform; storage/API delivery ≈ Load
- What comes next: lessons 02.1, 02.2, and 02.3 each go deep on one stage — Extractors, Transformers, and Loaders respectively

Transitions: ETL gives us the pattern. Now we need the components. Lesson 02.1 starts with the Extractor — the piece responsible for the E in ETL.

Key Principles:

- ETL is a separation of concerns: data acquisition, data processing, and data persistence are three distinct responsibilities
- Mixing ETL stages is the most common source of pipeline bugs
- Every component in a real pipeline maps to one of the three stages

Examples:

- E: a connector that pulls daily sales records from a PostgreSQL database
- T: a function that strips whitespace from product names, fills missing prices with 0, and computes a total (price × quantity)
- L: a writer that inserts the cleaned records into a data warehouse table with upsert logic

---

02.1 Extractors 🕮

Learning Objectives:

- Describe what an extractor does and what it must not do
- Identify common extractor types and their connection patterns
- Apply best practices for reliable, fault-tolerant extraction

Content Outline:

- What an extractor does: it connects to a data source and pulls data into the pipeline — nothing else
- The golden rule: extractors must not transform data; they return the raw data exactly as found
- Common extractor types:
    - File readers: CSV, JSON, Parquet from local disk or object storage
    - Database connectors: SQL queries, ORM-based readers
    - API clients: paginated REST calls, authenticated requests
    - Stream consumers: reading from a Kafka topic, a webhook, or a queue
- Key concerns when building extractors:
    - Connection handling: timeouts, retries, credential management
    - Pagination: fetching all records when the source paginates results
    - Rate limiting: respecting source API limits
    - Schema awareness: knowing what fields to expect and validating early
- Best practices (woven in): validate schema at extraction before passing data downstream; log how many records were extracted; implement retry with exponential backoff; make extractors stateless and independently testable
- Anti-patterns (woven in): calling an API and immediately transforming the result inside the extractor; no retry on transient network errors; ignoring pagination and silently missing records

Transitions: Once data is extracted, it is raw — and raw data is rarely usable. The next lesson covers Transformers: the stage that makes data clean, consistent, and meaningful.

Key Principles:

- Extractors are responsible for one thing: getting data — not cleaning it, not storing it
- Fault tolerance at extraction prevents data loss silently propagating through the pipeline
- An extractor that fails loudly is better than one that fails silently

Examples:

- A CSV extractor that reads a file, validates that the expected columns are present, logs the record count, and returns a list of raw dicts
- An API extractor that handles pagination by looping through all pages and collecting results before returning

---

02.2 Transformers 🕮

Learning Objectives:

- Describe what a transformer does and its defining properties
- Apply the most common transformation operations to a dataset
- Explain why transformers should be pure functions

Content Outline:

- What a transformer does: it takes raw data and returns clean, structured, enriched data — nothing else

- The golden rule: transformers must not read from sources or write to destinations; they are pure data-in, data-out functions
- Common transformation operations:
  - Cleaning: drop rows with null values, fill missing fields with defaults, fix encoding issues
  - Normalization: lowercase column names, strip whitespace, standardize date formats
  - Type casting: convert strings to numbers, parse ISO dates, coerce boolean fields
  - Enrichment: compute derived fields (e.g., total = price × quantity), join with a reference dataset
  - Filtering: drop records that fail validation rules
- Why pure functions matter: a transformer with no side effects is trivially testable, deterministic, and safe to run multiple times
- Best practices (woven in): write one transformation function per concern (single responsibility); test each transformer in isolation with sample data; never write to a database inside a transformer; log rejected records with the reason for rejection
- Anti-patterns (woven in): a transformer that also writes to a database mid-transformation; non-deterministic transforms (e.g., using `datetime.now()` inside the transform without injecting it); transformers that silently drop records without logging

Transitions: Clean, transformed data is ready to be persisted. The next lesson covers Loaders — the final stage that writes data to its destination.

Key Principles:

- Transformers are pure functions: same input always produces the same output
- Separating transformation from I/O is what makes pipelines testable and reliable
- Rejected records should be logged, not silently dropped

Examples:

- A transformer that receives a list of raw dicts with inconsistent column casing, strips and lowercases all keys, drops rows where `price` is null, and computes a `total` field
- A pandas-based transformer: `df.dropna() → df.rename(columns=str.lower) → df["total"] = df["price"] * df["quantity"]`

02.3 Loaders 🕮

Learning Objectives:

- Describe what a loader does and its key properties
- Identify common load destinations and their write modes
- Apply idempotency to loader design

Content Outline:

- What a loader does: it takes transformed data and writes it to the destination — database, file, API, data warehouse, message queue
- The golden rule: loaders must not transform data; they write what they receive
- Common load destinations:

    - Relational databases: INSERT, UPDATE, UPSERT (INSERT ON CONFLICT)
    - File storage: writing CSV/JSON/Parquet to disk or object storage
    - APIs: POSTing records to an external service
    - Data warehouses: bulk loads to BigQuery, Redshift, Snowflake
- Write modes:
    - Append: add records to existing data (use for event logs)
    - Overwrite: replace existing data entirely (use for full snapshots)
    - Upsert: insert if not exists, update if exists (use for entities with a primary key)
- Idempotency in loaders: a loader is idempotent if running it twice with the same data produces the same result — no duplicate records, no data loss
- Best practices (woven in): always use upsert over plain insert when the data has a natural key; validate data shape before writing; wrap writes in a transaction when possible; log how many records were loaded
- Anti-patterns (woven in): plain INSERT without deduplication logic (causes duplicates if the pipeline reruns); no transaction on writes (partial writes corrupt the destination); writing without logging (no audit trail)

Transitions: Now all three ETL components are in place. The next lesson is a hands-on challenge: build a basic pipeline that uses all three stages in Python.

Key Principles:

- Loaders are the final gate — write nothing you have not validated
- Idempotency is the difference between a pipeline that can be safely rerun and one that corrupts data on failure
- Write mode selection depends on the semantics of the data, not convenience

Examples:

- An upsert loader that inserts a sales record if the order ID does not exist, or updates the total if it does
- A file loader that writes a pandas DataFrame to a timestamped CSV file, then moves the previous version to an archive folder

---

02.4 Build a Basic Pipeline 💻

Challenge Prompt: Build a FastAPI endpoint that acts as a basic ETL pipeline: it receives a list of raw sales records, cleans and enriches the data using pandas and numpy, and returns the transformed records.

Solution Code:

```
import pandas as pd
import numpy as np
from fastapi import FastAPI

app = FastAPI()
```

```python
@app.post("/pipeline")
def run_pipeline(records: list[dict]) -> list[dict]:
    df = pd.DataFrame(records)
    df.columns = [col.lower().strip() for col in df.columns]
    df.dropna(subset=["price", "quantity"], inplace=True)
    df["price"] = pd.to_numeric(df["price"], errors="coerce")
    df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")
    df.dropna(inplace=True)
    df["total"] = np.round(df["price"] * df["quantity"], 2)
    return df.to_dict(orient="records")
```

Challenge Code:

```python
import pandas as pd
import numpy as np
from fastapi import FastAPI

app = FastAPI()

@app.post("/pipeline")
def run_pipeline(records: list[dict]) -> list[dict]:
    # 1. Load records into a DataFrame
    # 2. Normalize column names: lowercase and strip whitespace
    # 3. Drop rows where "price" or "quantity" is missing
    # 4. Convert "price" and "quantity" to numeric, drop rows that fail
    # 5. Add a "total" column: round(price * quantity, 2)
    # 6. Return the result as a list of dicts
    pass
```

03.0 Data Pipeline Assessment 🧠

Learning Objectives:

- Demonstrate understanding of data pipelines, ETL, and pipeline components
- Apply pipeline design principles to realistic scenarios
- Identify common pipeline anti-patterns and explain why they fail

Question Format: Scenario-based (choose the best action or identify the problem)

Questions:

1. A team runs a nightly pipeline that exports sales data to a warehouse. One morning, they notice the report is wrong — but the pipeline ran successfully with no errors. What is the most likely cause, and what pipeline property was missing?
    - A) The extract step failed silently — there was no schema validation at extraction
    - B) The transform step ran twice — the pipeline was not idempotent

- C) The load step used the wrong write mode — records were appended instead of upserted
- D) The pipeline ran in stream mode when it should run in batch
- **Correct: A** — silent failure at extraction due to no schema validation is the most common cause of pipelines that "succeed" but produce wrong results

2. You are building a transformer function that cleans a dataset. A colleague suggests calling the database inside the transformer to look up missing values. What is wrong with this approach?

    - A) Transformers should only use pandas — no external libraries
    - B) Transformers must be pure functions; adding I/O makes them non-deterministic, harder to test, and tightly coupled
    - C) The database call would be too slow for a transformer
    - D) Transformers cannot process missing values
    - **Correct: B**

3. Your pipeline's loader uses a plain INSERT statement. A network error causes the pipeline to rerun. What problem does this introduce, and how do you fix it?

    - A) The pipeline will fail — INSERT does not support reruns; use REPLACE instead
    - B) Duplicate records will appear in the destination — fix by using UPSERT with a natural key to make the loader idempotent
    - C) The INSERT will block — wrap it in a transaction
    - D) No problem — INSERT is idempotent by default
    - **Correct: B**

4. A startup processes 500 user events per day for their analytics dashboard. A developer proposes building a real-time streaming pipeline. What is the best response?

    - A) Agree — real-time is always more reliable than batch
    - B) Agree — streaming is the industry standard for event data
    - C) Disagree — at 500 events per day, a batch pipeline refreshing every hour is simpler, cheaper, and sufficient for the latency requirement
    - D) Disagree — streaming cannot handle user event data
    - **Correct: C**

5. In an ETL pipeline, when should data be validated?

    - A) Only at the load stage, just before writing to the destination
    - B) Only inside the transformer, since that is where data is cleaned
    - C) At the extractor (schema validation) and inside the transformer (business rules) — not at the loader
    - D) Validation is optional if the data source is trusted
    - **Correct: C**

6. A pipeline's extractor calls a paginated REST API. After deploying to production, the team notices the pipeline is only capturing the first page of results. What was missing?

    - A) Authentication credentials for the API

    - B) Retry logic for failed requests
    - C) Pagination handling — the extractor must loop through all pages before returning
    - D) A schema validation step
    - **Correct: C**

7. You have a transformer that adds a `processed_at` field using `datetime.now()`. A colleague says this transformer is not reliable. Why?

    - A) `datetime.now()` is not available in Python 3
    - B) Using `datetime.now()` inside the transformer makes it non-deterministic — the same input produces different output depending on when it runs, breaking reproducibility and testability
    - C) The transformer should use `datetime.utcnow()` instead
    - D) `processed_at` should be added by the loader, not the transformer
    - **Correct: B**

Evaluation Criteria:

- Scenarios test application of concepts, not recall of definitions
- Each question maps to at least one lesson in the course
- A passing score demonstrates the ability to reason about pipeline design, not just name ETL stages

Score Interpretation:

- 7/7: Ready to build production-grade pipelines
- 5-6/7: Solid foundations — review the lessons corresponding to missed questions
- Below 5: Revisit ETL and the components section before moving on

---

04.0 Pipelines in the Real World

Content Outline:

- Course recap: what students accomplished
    - Learned what a data pipeline is and why automation is non-negotiable
    - Understood batch vs. stream and when to use each
    - Mastered the ETL pattern and the three components that implement it (Extractor, Transformer, Loader)
    - Built a working pipeline endpoint in Python using FastAPI, pandas, and numpy
- Connection to bigger picture: everything that follows in this curriculum — telemetry collection, data formats, database selection — builds on the pipeline foundations established here
- What real pipelines look like: tools like Apache Airflow, Prefect, and dbt orchestrate exactly the components you just learned; the ETL pattern scales from a 20-line Python script to a system processing billions of events per day
- Next steps: Skill 40 covers how to choose the right data format and database for your pipeline's destination; Skills 41 and 42 go deep on telemetry — a major source of data that pipelines process

- Encouragement: you now think in pipelines — that mental model will be with you throughout the rest of the course and your career

Note: This is conclusion only — no theory, exercises, or assessment.